

Lab 2: 64-Bit XGMII Interface & Reconciliation Sublayer

Hardware-in-the-Loop Simulation, FCS CRC-32 & 64-Bit Waveform Analysis

Zuñiga, Ivan Andrés

August 25, 2026

Fundación Fulgor

1. System Architecture & 64-Bit XGMII Datapath

Hardware-in-the-Loop Setup:

- **Host Interface:** tap0 (10.0.0.1/24, MAC 02:00:00:00:00:01).
- **Target Interface:** tap1 in namespace ns_b (10.0.0.2/24, MAC 02:00:00:00:00:02).
- **C++ Testbench (wrapper.cpp):**
Non-blocking TAP I/O, converts L2 frames to 64-bit XGMII SDR words, appends IEEE 802.3 CRC-32 FCS, and checks FCS on RX.
- **RTL Core (top.sv):** Synchronous 8-lane parallel datapath (txd[63:0], txc[7:0]) with FSM header extraction and sliding-window FCS latching.
- **Standard:** IEEE 802.3 Clause 81 Reconciliation Sublayer (64-bit SDR MII mapping with /S/, /T/, and /I/).

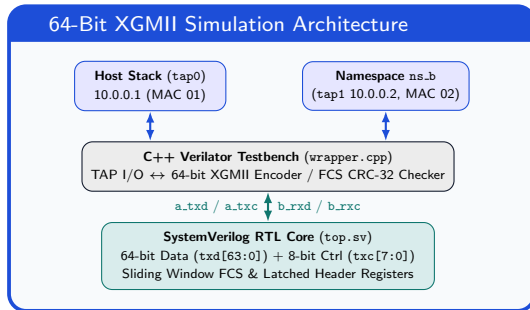


Figure 1: Simulation connecting Linux TAP network stacks to the 64-bit XGMII RTL pipeline.

2. Environment Setup & Traffic Generation

Execution Sequence:

1. TAP Network Initialization:

```
sudo ./setup_tap.sh
```

Configures tap0 and isolated namespace ns_b with tap1.

2. Verilator Build & Execution:

```
verilator --cc --trace --exe --build
```

```
wrapper.cpp top.sv
```

```
sudo ./obj_dir/emulator
```

3. Traffic Capture:

```
sudo tcpdump -i tap0 -w
```

```
lab2_capture.pcap
```

4. Pattern Traffic Injection:

```
ping -c 2 -p cafe0002 -I tap0 10.0.0.2
```

```
> sudo ./obj_dir/emulator
[TAP] Interface tap0 bound successfully (host).
[TAP] Interface tap1 bound successfully (ns_b).
[EMULADOR LAB 2] Running 64-bit XGMII emulator with FCS calculation & checking...
^C^C

> sudo ~/.local/bin/tcpdump -i tap0 -w lab2_capture.pcap
tcpdump: listening on tap0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C4 packets captured
4 packets received by filter
0 packets dropped by kernel

> ping -c 2 -p cafe0002 -I tap0 10.0.0.2
PATTERN: 0xcafe0002
PING 10.0.0.2 (10.0.0.2) from 10.0.0.1 tap0: 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=1.79 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=1.70 ms

--- 10.0.0.2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 1.700/1.745/1.790/0.045 ms
```

Figure 2: Terminal execution: TAP interface binding, Verilator build, and no loss in ICMP traffic.

3. Layer 2 / Layer 3 Wireshark Packet Analysis

Captured ICMP Echo Request Frame (98 Bytes Total):

- **Layer 2 Header (14B):** Dst MAC 02:00:00:00:00:02, Src MAC 02:00:00:00:00:01, EtherType 0x0800 (IPv4).
- **Layer 3 Header (28B):** IPv4 10.0.0.1 → 10.0.0.2 (20B, Len 84B, ID 0x7E53), ICMP Type 8 (Echo Request, Code 0, Checksum 0xECEC, ID 0x4702, Seq 1).
- **Payload (56B):** 16-byte kernel timestamp + 40-byte repeating pattern `ca fe 00 02` (offsets 0x003a–0x0061).

Frame 1: Packet, 98 bytes on wire (784 bits), 98 bytes captured	0000	02 00 00 00 00 02 02 00 00 00 00 01 08 00 45 00
Ethernet II, Src: 02:00:00:00:00:01 (02:00:00:00:00:01), Dst:	0010	00 54 7e 53 40 00 40 01 a8 53 0a 00 00 01 0a 00
Internet Protocol Version 4, Src: 10.0.0.1, Dst: 10.0.0.2	0020	00 02 08 00 ec ec 47 02 00 01 97 45 8b 6a 00 00
Internet Control Message Protocol	0030	00 00 a9 57 0a 00 00 00 00 00 ca fe 00 02 ca fe
	0040	00 02 ca fe 00 02 ca fe 00 02 ca fe 00 02 ca fe
	0050	00 02 ca fe 00 02 ca fe 00 02 ca fe 00 02 ca fe
	0060	00 02

Figure 3: Wireshark dissection and hex dump of the 98-byte captured ICMP frame.

4. XGMII Framing: Frame Start Delimiter & Preamble

Start Control Character & Preamble/SFD Lane Alignment (IEEE 802.3 Clause 81):

- **Start Delimiter** (/S/ = 0xFB): Aligned to Lane 0 with control bit TXC[0] = 1 (TXC = 8'h01).
- **Preamble & SFD Sequence**: 6 Preamble bytes (0x55) on Lanes 1–6, followed by 1 SFD byte (0xD5) on Lane 7 (TXD = 0xD5555555555555FB).

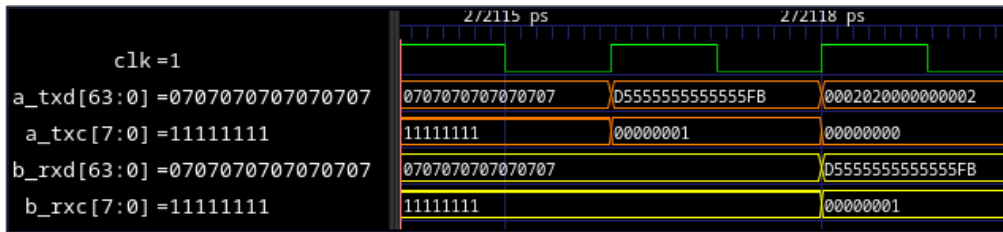


Figure 4: Transition from Idles (/I/ = 0x07) to Start of Frame (/S/ = 0xFB) on Lane 0.

5. SW-to-HW Correlation: Layer 2 & Layer 3 Headers

Parallel 64-Bit Word Mapping & RTL Register Latching (Cycles 1–3):

- **Cycle 1 (Dst/Src MAC):** TXD = 0x0002020000000002 → Lanes 0–5: Dst MAC 02:00:00:00:00:02, Lanes 6–7: Src MAC 02 00.
- **Cycle 2 (Src MAC/EtherType):** TXD = 0x0045000801000000 → Lanes 0–3: Src MAC 00 00 00 01, Lanes 4–5: EtherType 0x0800, Lanes 6–7: IPv4 45 00.
- **Cycle 3 (IPv4 Header):** TXD = 0x01400040537E5400 → Total Length 84B (0x54), ID 0x7E53, TTL 64 (0x40), Protocol 1 (0x01 ICMP).

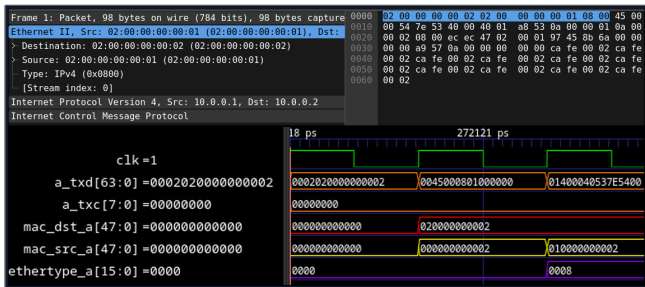


Figure 5: Cycle-by-cycle alignment between Wireshark L2/L3 hex and 64-bit lane distribution on `a_txd[63:0]`.

6. SW-to-HW Correlation: Multi-Lane Payload (0xCAFE0002)

ICMP Endpoints, Kernel Timestamp & Multi-Lane Payload Streaming:

- **Cycles 4–5 (IP Endpoints & ICMP Header):** Endpoints 10.0.0.1 → 10.0.0.2, ICMP Type 8 Code 0 (08 00), Checksum 0xECEC, and ID 0x4702.
- **Cycles 6–8 (16B Timestamp & Payload Transition):** Kernel timestamp across Cycles 6–8, with Lanes 2–7 of Cycle 8 initiating the ca fe 00 02 pattern (TXD = 0xFECA0200FECA0000).
- **Sustained Throughput:** Continuous 64-bit parallel transmission (8 bytes/cycle).

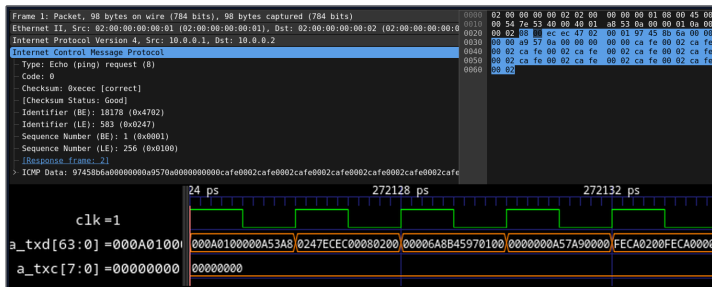


Figure 6: Multi-lane distribution of ICMP headers, timestamp, and 0xCAFE0002 payload across 64-bit XGMII cycles.

7. Frame Termination (/T/) & FCS (CRC-32) Validation

Frame Check Sequence (FCS) Appending & Control Delimiter Signaling (Cycle 13):

- **Data & FCS Field (Lanes 0–5):** Lanes 0–1 carry the final 2 payload bytes (00 02); Lanes 2–5 carry the 4-byte CRC-32 FCS (0x022EE9EC).
- **Control Delimiters (Lanes 6–7):** Lane 6 asserts Terminate (/T/ = 0xFD, TXC[6]=1); Lane 7 fills with Idle (/I/ = 0x07, TXC[7]=1), setting TXC = 8'hC0.

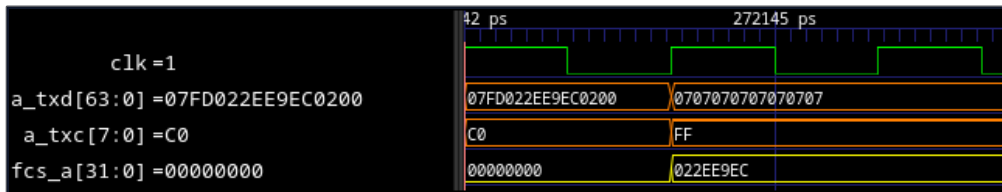


Figure 7: Cycle 13 frame termination: FCS CRC-32 (0x022EE9EC), and Terminate delimiter (/T/ = 0xFD).

8. Pipeline Latency & Register Timing Verification

RTL Synchronous Registered Datapath Verification (top.sv):

- **RTL Non-Blocking Assignment ($\leq$):** Flip-flop registers in top.sv ($b_rxd \leq a_txd$) use non-blocking assignments, creating the true physical 1-clock delay ($T_{\text{latency}} = 1 T_{\text{clk}}$).
- **Synchronous Testbench Timing:** wrapper.cpp drives stimulus synchronously on posedge clk to accurately model SystemVerilog launch-register timing.
- **Deterministic 1-Cycle Latency:** Confirms exact cycle-accurate pipeline delay between TX input and RX output registers.

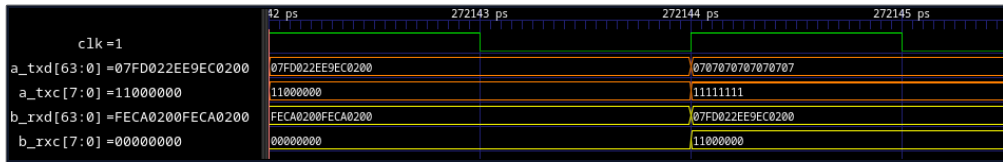


Figure 8: GTKWave trace: true 1-clock-cycle register latency ($1 T_{\text{clk}}$) through RTL non-blocking assignments.

Thank You!